Day 98: A Pretrained-Transformers Library, and a Real Pretrained Model

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 15, Day 1 | Complete study guide

Watch: Getting Started With Hugging Face in 15 Minutes (AssemblyAI) | What is BERT? AI Explained Simply (Step-by-Step) (Coursesteach)

Script: `day98_pretrained_transformers.py` · every number in this guide is copied by program from `day98_run_output.txt`, the real output of that script.

Weeks 11-14 built every piece of a Transformer by hand. Week 15 uses the tools practitioners use: the Hugging Face `transformers` library and models pretrained by others. Today's model, DistilBERT fine-tuned for sentiment (66,955,010 weights), gets 0.9106 of SST-2's 872 validation sentences right, against 0.8165 for a bag-of-words model written from scratch and trained on all 67,349 training items. It is also overconfident: its answers above 0.95 confidence average 0.997 but are right only 0.937 of the time.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

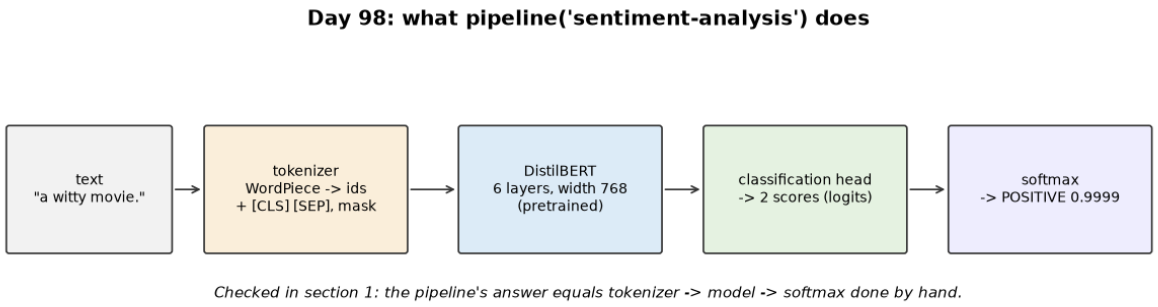
A **pretrained model** is a network someone else has already trained, usually on far more text than you have, and published with its weights. **BERT** (2018) is a Transformer encoder pretrained to fill in masked words; **DistilBERT** is a smaller copy trained to imitate it (6 layers instead of 12). Today's version was then fine-tuned to classify movie-review sentences as positive or negative.

The **transformers library** wraps each model in three objects: a **tokenizer** (text to ids), a **model** (ids to scores) and a **pipeline** (both, plus the last step, in one call).

REAL-WORLD SCENARIO

Buying a car instead of building one: you still need to know how to drive it, what it can carry, and how far to trust its fuel gauge.

SYSTEM DIAGRAM



What one pipeline call does, step by step; section 1 checks it by doing the steps by hand.

Today's part for Day 104

Week 15 keeps the two-track structure: Days 98-103 each add a short exercise isolating one part, and Day 104 builds a small retrieval-augmented question-answering system from them. `day98_exercise.py` is the batching part: `batched`, `run_batched` and a safe `softmax`. Its checks, real output:

ACTUAL OUTPUT

```
size  1: 10 chunks, sizes [1, 1, 1, 1, 1, 1, 1, 1, 1, 1], cover the list exactly in order: True
size  3: 4 chunks, sizes [3, 3, 3, 1], cover the list exactly in order: True
size  4: 3 chunks, sizes [4, 4, 2], cover the list exactly in order: True
size 10: 1 chunks, sizes [10], cover the list exactly in order: True
size 15: 1 chunks, sizes [10], cover the list exactly in order: True
softmax of [1000, 1001]: naive [nan nan], safe [0.268941 0.731059] (expected [0.268941, 0.731059])
batch 2: largest probability difference from one-at-a-time 6.0e-08
batch 4: largest probability difference from one-at-a-time 6.0e-08
batch 6: largest probability difference from one-at-a-time 8.9e-08
P(positive): 'a gorgeous, witty,' 1.000, 'the plot is nothin' 0.000, "it's fine." 1.000, 'not bad
    at all' 0.999, 'an utter waste of ' 0.000, 'I laughed, I cried' 0.150
```

Batching changes the model's probabilities only at the level of float rounding (under 1e-7), and the naive softmax returns NaN for large scores where the safe one does not. Note 'it's fine.' at 1.000 positive: a first sign of the overconfidence measured in section 3.

Setup and data

The libraries are installed with pip (PyTorch's CPU build, transformers, datasets). The model (about 268 MB) and SST-2 are downloaded on first use into a cache outside the repository and are not committed.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The library: tokenizer, model, pipeline

DEFINITION — start here, no prior knowledge assumed

`AutoTokenizer.from_pretrained(name)` and `AutoModelForSequenceClassification.from_pretrained(name)` download (once) and build the right classes from the model's saved configuration. `pipeline(...)` chains them and turns the two output scores into a label and a probability.

REAL-WORLD SCENARIO

A coffee machine's one button does grind, brew and pour; doing the three by hand should give the same cup.

MATHEMATICS

$$p(\text{class } k) = e^{z_k} / \Sigma_j e^{z_j} \quad (\text{softmax of the two logits } z), \quad \text{label} = \operatorname{argmax}_k p(\text{class } k)$$

IMPLEMENTATION

```
pipe = pipeline("sentiment-analysis", model=model, tokenizer=tok)
res = pipe(texts)
enc = tok(texts, padding=True, return_tensors="pt")
with torch.no_grad():
    logits = model(**enc).logits
probs = torch.softmax(logits, dim=-1).numpy()
```

ACTUAL OUTPUT

```
model: distilbert/distilbert-base-uncased-finetuned-sst-2-english
  DistilBertForSequenceClassification, 66,955,010 weights, 6 layers, width 768, 12 heads, labels {0: 'NEGATIVE', 1:
     'POSITIVE'}
tokenizer: BertTokenizer, vocabulary 30,522, longest input 512 tokens
  'a gorgeous, witty, seductive movie.'
    pipeline: POSITIVE 0.999888 | by hand: logits [-4.368  4.727], softmax 0.999888
  'the plot is nothing but boilerplate cliches.'
    pipeline: NEGATIVE 0.999816 | by hand: logits [ 4.769 -3.831], softmax 0.999816
largest gap between the pipeline and tokenizer -> model -> softmax: 0.0e+00
Actual output from this lesson's run.
```

What it means

The pipeline is exactly tokenizer → model → softmax: the largest gap is 0.0e+00. The model has 6 layers (width 768, 12 heads) and 66,955,010 weights, about 80 times Day 92's mini-GPT. The scores are far apart (for example -4.368 and 4.727), so the probabilities sit very close to 0 or 1.

WHY CHOOSE THIS?

One line gives a working classifier, with the same answer as the manual steps.

WHY NOT THIS? (TRADEOFFS)

The convenience hides the steps; knowing them is what lets you batch, debug and measure.

2. The tokenizer: WordPiece, special tokens, padding

DEFINITION — start here, no prior knowledge assumed

BERT's tokenizer is **WordPiece**: a fixed vocabulary of 30,522 pieces, where a word not in it is split into known pieces, continuation pieces marked ##. Every input gets `[CLS]` at the start (its final vector feeds the classifier) and `[SEP]` at the end. In a batch, shorter inputs are padded with `[PAD]`, and the **attention mask** (1 = real, 0 = padding) stops attention from looking at the padding.

REAL-WORLD SCENARIO

A dictionary that lists 'un-', 'watch' and '-able' separately can still spell 'unwatchable'.

IMPLEMENTATION

```
alone = predict_proba(tok, model, X[:64], bs=1)
padded = predict_proba(tok, model, X[:64], bs=64)
print(f"64 sentences one at a time vs padded into one batch: largest probability difference
    {np.abs(alone - padded).max():.1e}")
```

ACTUAL OUTPUT

```
  "The movie wasn't unwatchable, just forgettable." -> ['the', 'movie', 'wasn', "'", 't', 'un', '##watch', '##able', ',',
      'just', 'forget', '##table', '.']
  "Spielberg's best since Jaws" -> ['spielberg', "'", 's', 'best', 'since', 'jaws']
three validation sentences padded into one batch (ids and attention mask of the shortest):
  ids  [101, 4895, 10258, 2378, 8450, 2135, 21657, 1998, 7143, 102, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
  mask [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
  [CLS] = 101, [SEP] = 102, [PAD] = 0
over all 872 validation sentences: 17,046 words -> 20,199 pieces (1.185 per word); 0.118 of words split into more than one
    piece; [UNK] pieces: 0
64 sentences one at a time vs padded into one batch: largest probability difference 1.4e-06
Actual output from this lesson's run.
```

What it means

Real review text needs few pieces: 1.185 pieces per word, and only 0.118 of words are split; there were [UNK] pieces: 0 on the whole validation set, because any word can be spelled from pieces. The split words are the rare ones ('un ##watch ##able', 'forget ##table'), and the uncased tokenizer lowercases everything ('spielberg'). Padding is harmless: one-at-a-time and padded predictions differ by at most 1.4e-06, float rounding, because the mask removes the padding from attention.

WHY CHOOSE THIS?

No unknown words, and a vocabulary small enough for a 768-wide embedding table.

WHY NOT THIS? (TRADEOFFS)

Rare words cost several positions of the 512-token limit, and an uncased model cannot see capitals.

3. Accuracy and calibration on real data

DEFINITION — start here, no prior knowledge assumed

Accuracy on SST-2's 872 validation sentences, with a 95% **bootstrap** interval: resample the 872 right/wrong results with replacement 2,000 times and take the middle 95% of the resampled accuracies. **Calibration** asks whether a confidence of 0.9 means right 90% of the time: answers are grouped into confidence bins, and the **expected calibration error** (ECE) is the bin-size-weighted gap between confidence and accuracy.

REAL-WORLD SCENARIO

A weather forecaster who says '90% chance of rain' should see rain on about nine of every ten such days.

MATHEMATICS

$$ECE = \Sigma_{bins\ b}\ (n_b / N) \cdot |\ mean\ confidence_b - accuracy_b\ |$$

95% interval = 2.5th and 97.5th percentiles of accuracy over 2,000 resamples of the N results

IMPLEMENTATION

```
def bootstrap_ci(correct, n_boot=N_BOOT, seed=0):
    rng = np.random.RandomState(seed)
    idx = rng.randint(0, len(correct), (n_boot, len(correct)))
    accs = correct[idx].mean(axis=1)
    return np.percentile(accs, 2.5), np.percentile(accs, 97.5)
def calibration(conf, correct, n_bins=10):
    edges = np.linspace(0.5, 1.0, n_bins + 1)
    rows, ece = [], 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi) if lo > 0.5 else (conf >= lo) & (conf <= hi)
        if m.sum() == 0:
            continue
        rows.append((lo, hi, m.sum(), conf[m].mean(), correct[m].mean()))
        ece += m.mean() * abs(conf[m].mean() - correct[m].mean())
    return rows, ece
```

ACTUAL OUTPUT

```
872 sentences (444 positive, 428 negative) in 19.2 s with 4 threads
accuracy 0.9106, 95% bootstrap interval [0.8911, 0.9289] (2000 resamples)
confusion matrix (rows true neg/pos, columns predicted neg/pos): [[381, 47], [31, 413]]
  negative: precision 0.925, recall 0.890
  positive: precision 0.898, recall 0.930
calibration: answers grouped by confidence
    confidence  answers  mean conf  accuracy
  0.50 - 0.55        4      0.526     1.000
  0.55 - 0.60        1      0.592     1.000
  0.60 - 0.65        5      0.637     0.800
  0.65 - 0.70        4      0.668     0.500
  0.70 - 0.75        4      0.720     0.750
  0.75 - 0.80        7      0.775     0.571
  0.80 - 0.85        7      0.833     0.714
  0.85 - 0.90        8      0.869     0.375
  0.90 - 0.95       26      0.931     0.500
  0.95 - 1.00      806      0.997     0.937
expected calibration error: 0.0803; answers with confidence above 0.99: 0.844 of all
Actual output from this lesson's run.
```

What it means

Accuracy is 0.9106, interval [0.8911, 0.9289]: with 872 sentences, about two points either way. Errors are slightly lopsided: it misses more negatives (47) than positives (31). Its scores are a caveat, not a clean test: the model's authors fine-tuned on SST-2's training set and may have used this validation set to choose settings.

It is overconfident. 0.844 of its answers have confidence above 0.99. The top bin holds 806 answers with mean confidence 0.997 but accuracy 0.937; the 26 answers near 0.93 are right only 0.500 of the time. ECE is 0.0803. A 0.999 from this model is not a 0.1% chance of being wrong.

WHY CHOOSE THIS?

An interval and a calibration check turn one accuracy number into something you can rely on.

WHY NOT THIS? (TRADEOFFS)

Calibration bins with a handful of answers are very noisy; only the large bins say much.

4. What pretraining buys: a from-scratch baseline

DEFINITION — start here, no prior knowledge assumed

A **bag-of-words** logistic regression: each word in the vocabulary has one weight, a sentence's score is the sum of its words' weights plus a bias, and P(positive) is the sigmoid of the score. Written in NumPy and trained by full-batch Adam on 1,000, 10,000 and all 67,349 SST-2 training items, then scored on the same 872 sentences.

REAL-WORLD SCENARIO

Judging a review by counting good and bad words, without reading how they are put together.

MATHEMATICS

$$s(x) = b + \Sigma_{w \text{ in } x}\, \theta_w, \quad P(\text{positive} \mid x) = 1 / (1 + e^{-s(x)})$$

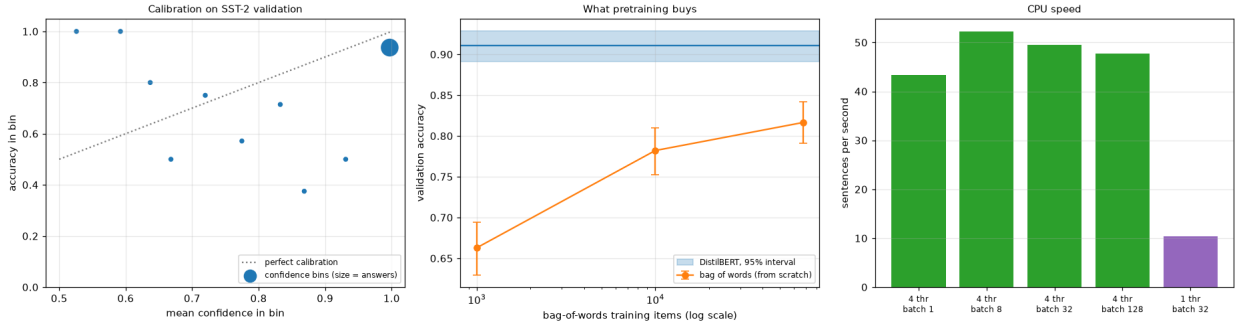
$$\text{loss} = -(1/N)\, \Sigma\, [y \log p + (1 - y) \log(1 - p)] + \lambda\, ||\theta||^2 / 2$$

IMPLEMENTATION

```
def fit(self, texts, y, steps=300, lr=0.05, l2=1e-4):
    ids, rows = self.encode(texts); n = len(texts)
    self.w, self.b = np.zeros(len(self.vocab)), 0.0
    m, v = np.zeros(len(self.vocab) + 1), np.zeros(len(self.vocab) + 1)
    for t in range(1, steps + 1):
        p = 1 / (1 + np.exp(-self.scores(ids, rows, n)))
        g_s = (p - y) / n
        g = np.r_[np.bincount(ids, weights=g_s[rows], minlength=len(self.vocab)) + l2 * self.w,
            g_s.sum()]
        m = 0.9 * m + 0.1 * g; v = 0.999 * v + 0.001 * g ** 2
        step = lr * (m / (1 - 0.9 ** t)) / (np.sqrt(v / (1 - 0.999 ** t)) + 1e-8)
        self.w -= step[:-1]; self.b -= step[-1]
    return self
```

ACTUAL OUTPUT

```
Logistic regression on which words occur (a word counts if it appears in at least 2 training texts), 300 full-batch Adam
    steps, written in NumPy. SST-2's training set has 67,349 items, many of them short phrases cut from the sentences; mean
    length 9.4 words, against 19.5 in validation.
 training items  vocabulary  accuracy        95% interval
          1,000         962    0.6628      [0.6296, 0.6938]
         10,000       6,419    0.7821      [0.7523, 0.8096]
         67,349      13,371    0.8165      [0.7913, 0.8417]
     DistilBERT      30,522    0.9106      [0.8911, 0.9289]
on the same sentences (bag of words on all items): both right 0.771, only DistilBERT right 0.140, only bag of words right
    0.046
Actual output from this lesson's run.
```



Left: calibration bins (dot size = answers). Middle: the bag-of-words model against training size, with DistilBERT's 95% interval. Right: CPU speed by threads and batch size.

What it means

More data helps the counting model (0.6628, 0.7821, 0.8165), but even with every training item it stays clearly below DistilBERT (0.9106); the intervals do not overlap. On the same sentences DistilBERT alone is right 0.140 of the time where the bag of words is wrong, and the reverse happens 0.046 of the time. Two cautions: SST-2's training items are mostly short phrases (9.4 words on average against 19.5 in validation), which suits neither model perfectly, and DistilBERT had the same labels plus its pretraining, so this measures pretraining plus a better model together.

WHY CHOOSE THIS?

The baseline is tiny, fast and fully understandable; it shows how much is left for the big model to add.

WHY NOT THIS? (TRADEOFFS)

A stronger classical baseline (word pairs, tuned regularisation) would close some of the gap; not tested.

5. Where it fails

DEFINITION — start here, no prior knowledge assumed

The 78 mistakes, sorted by how confident the model was, and accuracy split by two simple properties: whether the sentence contains a negation word, and its length.

REAL-WORLD SCENARIO

Reading a student's most confident wrong answers tells you more than their average mark.

IMPLEMENTATION

```
neg = np.array([bool(re.search(r"\b(not|no|never|n't)\b|n't", s)) for s in X])
```

ACTUAL OUTPUT

```
78 mistakes; the five made with the highest confidence:
  true negative, said the other with 0.9998: "it 's somewhat clumsy and too lethargically paced -- but its story about a
      mysterious creature with psychic ab"
  true negative, said the other with 0.9997: "although huppert 's intensity and focus has a raw exhilaration about it , the
      piano teacher is anything but fu"
  true negative, said the other with 0.9996: "the lower your expectations , the more you 'll enjoy it . "
  true negative, said the other with 0.9996: "the longer the movie goes , the worse it gets , but it 's actually pretty good
      in the first few minutes . "
  true positive, said the other with 0.9994: "if steven soderbergh 's ` solaris ' is a failure it is a glorious failure . "
with a negation word (not, no, never, n't): 164 sentences, accuracy 0.866; without: 708, accuracy 0.921
  short  sentences (2-15 words, 311 of them): accuracy 0.916
  middle sentences (16-23 words, 286 of them): accuracy 0.909
  long   sentences (24-47 words, 275 of them): accuracy 0.905
95% intervals: negation [0.811, 0.915], no negation [0.901, 0.941]
Actual output from this lesson's run.
```

What it means

The most confident mistakes are mixed reviews: a criticism followed by praise ('somewhat clumsy ... but'), contrast ('the longer the movie goes, the worse it gets, but ...'), or irony ('a glorious failure', 'the lower your expectations, the more you'll enjoy it'), all at confidence 0.9994 or above. Sentences with a negation word are harder (0.866 against 0.921), though the intervals overlap slightly ([0.811, 0.915] and [0.901, 0.941]): likely a real effect, not proven by this sample. Length barely matters (0.916, 0.909, 0.905).

WHY CHOOSE THIS?

Concrete failure cases suggest what to test next, such as contrast and irony.

WHY NOT THIS? (TRADEOFFS)

A few dozen examples per group: small differences are not significant.

6. Speed on a CPU

DEFINITION — start here, no prior knowledge assumed

Sentences per second on the same 128 sentences: batch sizes 1, 8, 32 and 128 with 4 PyTorch threads, and batch 32 with 1 thread; median of three timings.

REAL-WORLD SCENARIO

A bus carries more people per trip, but a full bus has to wait for the slowest passenger to board.

IMPLEMENTATION

```
for th, bs in ((THREADS, 1), (THREADS, 8), (THREADS, 32), (THREADS, 128), (1, 32)):
    torch.set_num_threads(th)
    ts = []
    for _ in range(3):
        t0 = time.perf_counter(); predict_proba(tok, model, texts, bs=bs);
            ts.append(time.perf_counter() - t0)
    res[(th, bs)] = len(texts) / np.median(ts)
```

ACTUAL OUTPUT

```
Sentences per second on the first 128 validation sentences; median of 3 timings each. Timings on
    this machine vary from run to run, so only the comparisons within this run mean anything.
  4 thread(s), batch   1:   43.4 sentences per second
  4 thread(s), batch   8:   52.3 sentences per second
  4 thread(s), batch  32:   49.5 sentences per second
  4 thread(s), batch 128:   47.7 sentences per second
  1 thread(s), batch  32:   10.4 sentences per second
Actual output from this lesson's run.
```

What it means

Batching helps a little and then hurts: 43.4 sentences per second one at a time, 52.3 in batches of 8, 47.7 in one batch of 128. Every sentence in a batch is padded to the longest, so big batches of mixed lengths do wasted work. Threads matter far more: 10.4 with one thread against 49.5 with four. These are one run's timings; an earlier run of the same script gave a different value for batch 128, so only the large differences should be trusted.

WHY CHOOSE THIS?

Moderate batches and several threads are an easy speed-up on a CPU.

WHY NOT THIS? (TRADEOFFS)

Timing on a shared laptop is noisy; sorting inputs by length would cut padding further (not tested).

Interview preparation — technical questions

Q: What does a transformers pipeline do?

A: Tokenize, run the model, and turn logits into labels with softmax; here identical to the manual steps (gap 0).

Q: What are [CLS], [SEP] and the attention mask?

A: Start and end tokens (the [CLS] vector feeds the classifier) and a 1/0 mask that hides padding from attention.

Q: Why does padding not change predictions?

A: The mask excludes padded positions; one-at-a-time and padded results differed by 1.4e-06.

Q: How do you put an interval on an accuracy?

A: Bootstrap: resample the per-example results; here [0.8911, 0.9289].

Q: What is expected calibration error?

A: The bin-weighted gap between confidence and accuracy; 0.0803 here.

Interview preparation — theoretical questions

Q: Why does WordPiece never produce unknown words?

A: Its vocabulary includes single characters and pieces, so any word can be spelled from them.

Q: Why can answers at 0.997 mean confidence be wrong 6% of the time?

A: A common explanation: cross-entropy training keeps pushing scores apart, and nothing forces probabilities to match frequencies.

Q: What is DistilBERT?

A: A 6-layer student trained to imitate 12-layer BERT; smaller and faster, slightly less accurate.

Q: Why is SST-2 validation not a clean test here?

A: It is the standard development set; the model's authors may have tuned on it.

Key takeaway and what's next

> **KEY TAKEAWAY**
> A pretrained model is a few lines away, and the pipeline is nothing more than tokenizer, model and softmax. Measured honestly on real data it is strong (0.9106, interval 0.8911 to 0.9289, against 0.8165 for a from-scratch bag of words) but overconfident (ECE 0.0803), weakest on mixed and ironic reviews, and fastest on a CPU with several threads and moderate batches.

- 66,955,010 weights; 30,522-piece WordPiece vocabulary; 1.185 pieces per word, no [UNK].
- Accuracy 0.9106; bag of words 0.6628 / 0.7821 / 0.8165 with 1,000 / 10,000 / 67,349 items.
- Top confidence bin: 806 answers, confidence 0.997, accuracy 0.937.
- Negation 0.866 vs 0.921 without; speed 52.3 sentences per second at batch 8, 10.4 with one thread.

Suggested watch order, if starting from zero

- 1. What is BERT? AI Explained Simply (Step-by-Step) (Coursesteach) — what BERT is and how it is pretrained.
- 2. Getting Started With Hugging Face in 15 Minutes (AssemblyAI) — pipelines, tokenizers and models in the library.

Day 99 preview

- **Fine-tuning a BERT-style model ourselves.** Plain DistilBERT, with no sentiment training, fine-tuned on part of SST-2's training set on this CPU, and compared with today's ready-made model and the bag-of-words baseline.